

Loader Animation

Build a fullscreen loading screen component in React (Next.js 14, TypeScript). Uses Framer Motion for animations. Here is the exact specification:

Theme

CSS

```
--bg: #0a0a0a;  
--text: #f5f5f5;  
--muted: #888888;  
--stroke: #1f1f1f;
```

Fonts: font-display → Instrument Serif (Google Fonts, italic, weight 400).

Component: LoadingScreen

Receives one prop: onComplete: () => void.

Container: <motion.div> — fixed inset-0 z-[9999] bg-bg. Exit animation: exit={{ opacity: 0 }}, duration 0.6s, ease [0.4, 0, 0.2, 1]. Wrap in <AnimatePresence mode="wait"> from the parent.

Element 1: "Portfolio" Label (Top-Left)

<motion.div> — absolute top-8 left-8 md:top-12 md:left-12.

Text: "Portfolio"

Class: text-xs md:text-sm text-muted uppercase tracking-[0.3em]

Entrance animation: initial={{ opacity: 0, y: -20 }}, animate={{ opacity: 1, y: 0 }}, duration 0.6s, delay 0.1s

Element 2: Rotating Words (Center)

absolute inset-0 flex items-center justify-center.

Three words cycle in sequence: "Design" → "Create" → "Inspire". A new word appears every 900ms. The word index increments via setInterval and stops at the last word (doesn't loop).

Each word is a <motion.span> inside <AnimatePresence mode="wait">, keyed by wordIndex:

Class: text-4xl md:text-6xl lg:text-7xl font-display italic text-text/80

initial={{ opacity: 0, y: 20 }}

animate={{ opacity: 1, y: 0 }}

exit={{ opacity: 0, y: -20 }}

transition={{ duration: 0.4, ease: [0.4, 0, 0.2, 1] }}

Element 3: Counter (Bottom-Right)

<motion.div> — absolute bottom-8 right-8 md:bottom-12 md:right-12.

A number that counts from 000 → 100 over exactly 2.7 seconds using requestAnimationFrame. Each frame calculates elapsed / 2700 * 100. The number is displayed zero-padded to 3 digits (e.g. 007, 042, 100):

{Math.round(progress).toString().padStart(3, '0')}

Class: text-6xl md:text-8xl lg:text-9xl font-display text-text tabular-nums

Entrance animation: initial={{ opacity: 0, y: 20 }}, animate={{ opacity: 1, y: 0 }}, duration 0.6s, delay 0.1s

When progress reaches 100: Wait 400ms, then call onComplete(). Use a ref for onComplete to avoid stale closures.

Element 4: Progress Bar (Bottom Edge)

absolute bottom-0 left-0 right-0. A 3px tall track:

Track: `h-[3px] bg-stroke/50 (full width)`

Fill: `<motion.div>` inside the track:

`h-full origin-left`

Background: `linear-gradient(90deg, #89AACC 0%, #4E85BF 100%)`

Glow: `boxShadow: "0 0 8px rgba(137, 170, 204, 0.35)"`

`initial={{ scaleX: 0 }}`

`animate={{ scaleX: progress / 100 }}`

`transition={{ duration: 0.1, ease: "linear" }}`

Parent Wrapper Behavior

The parent component (AppWrapper) controls visibility:

State: `isLoading` starts true

Renders `<LoadingScreen onComplete={() => setIsLoading(false)} />` inside `<AnimatePresence mode="wait">` only when `isLoading` is true

Main page content sits below with: `style={{ opacity: isLoading ? 0 : 1, transition: "opacity 0.5s ease-out" }}`

When the loader calls `onComplete`, it triggers: loader fades out (0.6s) → page fades in (0.5s)

Timing Summary

0.0s — Loader appears, "Portfolio" slides in, counter starts at 000

0.0s — "Design" appears

0.9s — "Create" replaces "Design"

1.8s — "Inspire" replaces "Create"

2.7s — Counter hits 100, progress bar full

3.1s — `onComplete` fires (400ms delay)

3.1s — Loader fades out (0.6s exit animation)

3.7s — Page content fades in (0.5s opacity transition)